Sahrdaya College of Engineering and Technology, Kodakara			
Answer Key			
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY SECOND SEMESTER B.TECH DEGREE EXAMINATION, JULY 2021			
Course Code: EST 102			
Max Marks:100			**F.N Session**
Course Name: PROGRAMMING IN C(COMMMON TO ALL PROGRAMS)			
PART A			
		Answer all questions. Each question carries 3 marks	Marks
Q1		Differentiate between system software and application software.	
	ans	1. System Software is the type of software which is the interface between application software and system. On other hand Application Software is the type of software which runs as per user request. It runs on the platform which is provided by system software. 2. System software is used for operating computer hardware. On the other hand Application software is used by users to perform specific tasks. 3. Some examples of system software are compiler, assembler, debugger, driver, etc. Some examples of application software's are word processor, web browser, media player, etc	1 1 1
Q2		Differentiate between compiler and interpreter.	
	ans	Interpreter translates just one statement of the program at a time into machine code. Compiler scans the entire program and translates the whole of it into machine code at once. Interpreters are slow whereas compilers are fast translators.	1.5 1.5
Q3		What is the importance of precedence and associativity? Write the table for operator precedence.	

<table>
<tr><td></td><td>ans</td><td>Precedence: specifies the order of evaluation of operators in an expression.
Associativity: It defines the order in which operators of the same precedence are evaluated in an expression. Associativity can be either from left to right or right to left.

<table>
<tr><th>Operator category</th><th>Operators</th><th>Associativity</th></tr>
<tr><td>unary operators</td><td>- ++ -- ! sizeof (type)</td><td>R → L</td></tr>
<tr><td>arithmetic multiply, divide and remainder</td><td>* / %</td><td>L → R</td></tr>
<tr><td>arithmetic add and subtract</td><td>+ -</td><td>L → R</td></tr>
<tr><td>relational operators</td><td>< <= > >=</td><td>L → R</td></tr>
<tr><td>equality operators</td><td>== !=</td><td>L → R</td></tr>
<tr><td>logical and</td><td>&&</td><td>L → R</td></tr>
<tr><td>logical or</td><td>||</td><td>L → R</td></tr>
<tr><td>conditional operator</td><td>? :</td><td>R → L</td></tr>
<tr><td>assignment operators</td><td>= += -= *= /= %=</td><td>R → L</td></tr>
</table></td><td>1
1

1</td></tr>
<tr><td>Q4</td><td></td><td>Differentiate between ‘break’ and ‘continue’ statements.</td><td></td></tr>
<tr><td></td><td>ans</td><td>The main difference between break and continue is that break is used for immediate termination of a loop. On the other hand, ‘continue’ terminates the current iteration and resumes the control to the next iteration of the loop.
Example for break:
for (int j = 0; j < 5; j++) {

// Break Statement
if (j == 2)
break;

printf("%d ", j);
}

output: 0 1

Example for continue:
for (int j = 0; j < 5; j++) {

// Continue Statement
if (j ==2)
continue;

printf("%d ", j);
}
output: 0 1 3 4</td><td>1

1

1</td></tr>
<tr><td>Q5</td><td></td><td>Explain any 3 string handling functions using examples.</td><td></td></tr>
</table>

	ans	1. strlen()-strlen() function is used to find the length of string. Length of string means number of characters in the string.
		strlen() function accepts a string as an argument and returns an integer number which indicates length of string.
		Syntax
		length = strlen (StringName);
		2. strcpy()- strcpy() function is used to copy one string into another string.
		strcpy() function accepts two strings (source and destination) as an argument and copies source string into destination string. It does not return any value.
		Syntax:
		strcpy(DestinationString, SourceString);
		3.strcat()- strcat() function is used to append (join) one string at the end of another string.
		strcat function accepts two strings (string1 and string2) as an argument and appends string2 at the end of string1 and the result is stored in string1. It does not return any value.
		Syntax:
		strcat (String1, String2);
			1
			
			1
			
			1

Q6		Write a C program to find the occurrence of each element in an array.	

	ans	<pre>#include <stdio.h>
		int main ()
		{
		 int arr[100], freq[15];
		 int size, i, j, count;
		 printf("Enter size of array: ");
		 scanf("%d", &size);
		 printf("Enter elements in array: ");
		 for(i=0; i<size; i++)
		 {
		 scanf("%d", &arr[i]);
		 /* Initially initialize frequencies to -1 */
		 freq[i] = -1;
		 }
		 for(i=0; i<size; i++)
		 {
		 count = 1;
		 for(j=i+1; j<size; j++)
		 {
		 /* If duplicate element is found */
		 if(arr[i]==arr[j])
		 {
		 count++;
		 /* Make sure not to count frequency of same element again */
		 freq[j] = 0;
		 }
		 }
		 /* If frequency of current element is not counted */
		 if(freq[i] != 0)
		 {
		 freq[i] = count;
		 }
		 }
		 /*
		
		 printf("\nFrequency of all elements of array : \n");
		 for(i=0; i<size; i++)
		 {
		 if(freq[i] != 0)
		(3)
		01EST102052001 A 3
		 {
		 printf("%d occurs %d times\n", arr[i], freq[i]);
		 }
		 }
		 return 0;
		}</pre>
			3

Q7		Define formal parameters and actual parameters. Illustrate with an example.	

	ans	Actual Parameters are the values that are passed to the function when it is invoked while Formal Parameters are the variables defined by the function that receives values when the function is called.
		1. int sum(int a, int b) //Statement 1
		2. {
		3. return a+b;
		4. }
		5. int main()
		6. {
		7. int x=10,y=20;
		8. int s = sum(x,y); //Statement 2
		9. return 0;
		   }
		In Statement 1, the variables a and b are called FORMAL PARAMETERS.
		In Statement 2 the arguments x and y are called ACTUAL PARAMETERS
			1.5
			
			1.5

Q8		With examples show how: (i) an array is passed as an argument of a function. (ii) individual elements of an array are passed as arguments of a function.	

	ans	(i) an array is passed as an argument of a function.
		The array name itself is the address of the first element of that array. For example if the array name is arr then you can say that **arr** is equivalent to the **&arr[0]**.
		Example:
		#include<stdio.h>
		int sum(int num[]);
		int main()
		{
		int result,num[]=10,20,30,40,50};
		result=sum(num); //num array is passed to sum
		printf("Result=%d",result);
		return 0;
		}
		int sum(int num[])
		{
		int i,total=0;
		for(int i=0;i<6;i++)
		{
		sum=sum+num[i];
		}
		return sum;
		}
		
		(ii) individual elements of an array are passed as arguments of a function.
		#include<stdio.h>
		void print(int n);
		int main()
		{
		int result,num[]={10,20,30,40,50};
		print(num[0]); //num array is passed to sum
		}
		void print(int n)
		{
		printf("number at 0th position=%d",n);
		}
			1.5
			
			1.5

Q9		Write any three file handling functions in C.	

	ans	1.fopen(): The library function fopen() is used to open a file. This function is typically written as ptvar= fopen(file-name, file-type); where file-name and file- type are strings that represent the name of the data file and the manner in which the data file will be utilized. The name chosen for the file -name must be consistent with the rules for naming files, as determined by the computer's operating system. The file- type must be "r","w","a","r+","w+","a+". 2.fputc() It is the simplest function to **write individual characters** to a stream − **int fputc(int c, FILE *fp);** The function fputc() writes the character value of the argument c to the output stream referenced by fp. It returns the written character written on success otherwise EOF if there is an error. 3.fgetc() It is the simplest function **to read a single character** from a file − **int fgetc(FILE * fp);** The fgetc() function reads a character from the input file referenced by fp. The return value is the character read, or in case of any error, it returns EOF.	
Q10		Differentiate between address operator(&) and indirection(*) operator.	
	ans	The operator & and the immediately preceding variable returns the address of the variable associated with it. Indirection operator (*) also called as value at address. It returns a value stored at that address.	
PART B			
Answer any one question from each module. Each question carries 14 marks			
Module 1			
Q11		a) Explain different types of memory used in a computer. (7) b) Write an algorithm to find sum of digits of a number. (7)	

	ans	

a) Computer memory is of two basic types –

Primary memory(RAM and ROM) and Secondary memory (hard drive, CD, etc). Random Access Memory (RAM) is primary-volatile memory and Read Only Memory (ROM) is primary-non-volatile memory.

Random Access Memory (RAM) is used to store the programs and data being used by the CPU in real-time. The data on the random access memory can be read, written, and erased any number of times. RAM is a hardware element where the data being currently used is stored. It is a volatile memory. Types of RAM:

1. Static RAM, or (SRAM) which stores a bit of data using the state of a six transistor memory cell.
2. Dynamic RAM, or (DRAM) which stores a bit data using a pair of transistor and capacitor which constitute a DRAM memory cell.

Read Only Memory (ROM) is a type of memory where the data has been prerecorded. Data stored in ROM is retained even after the computer is turned off ie, non-volatile. Types of ROM:

1. Programmable ROM, where the data is written after the memory chip has been created. It is non-volatile.
2. Erasable Programmable ROM, where the data on this non-volatile memory chip can be erased by exposing it to high-intensity UV light.
3. Electrically Erasable Programmable ROM, where the data on this non-volatile memory chip can be electrically erased using field electron emission.
4. Mask ROM, in which the data is written during the manufacturing of the memory chip.

b) Algorithm to find sum of digits of a number

1. Input a Number
2. Initialize Sum to zero
3. While Number is not zero
4. Get Remainder by Number Mod 10
5. Add Remainder to Sum
6. Divide Number by 10
7. Print sum

Module 2		
Q12		Explain bubble sort with an example. Draw a flowchart and write pseudo code to perform bubble sort on an array of numbers

ans

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

Example:

First Pass:

(5 1 4 2 8) –> (1 5 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.

(1 5 4 2 8) –> (1 4 5 2 8), Swap since 5 > 4

(1 4 5 2 8) –> (1 4 2 5 8), Swap since 5 > 2

(1 4 2 5 8) –> (1 4 2 5 8), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:

(1 4 2 5 8) –> (1 4 2 5 8)

(1 4 2 5 8) –> (1 2 4 5 8), Swap since 4 > 2

(1 2 4 5 8) –> (1 2 4 5 8)

(1 2 4 5 8) –> (1 2 4 5 8)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one whole pass without any swap to know it is sorted.

Third Pass:

(1 2 4 5 8) –> (1 2 4 5 8)

(1 2 4 5 8) –> (1 2 4 5 8)

(1 2 4 5 8) –> (1 2 4 5 8)

(1 2 4 5 8) –> (1 2 4 5 8)

Algorithm for Bubble Sort

Step 1: Start

Step 2: Read the array of given items from the user.

Step 3: Take the first element(index = 0), compare the current element with the next element.

Step 4: If the current element is greater than the next element, swap them.

Step 5: Else,If the current element is less than the next element, then move to the next element.

Step 6: Repeat Step 3 to Step 5 until all elements are sorted.

Q13		a) Explain different data types supported by C language with their memory requirements. b) Write a C program to check if a number is present in a given list of numbers. If present, give location of the number otherwise insert the number in the list at the end.	

ans

a) **int or signed int-2bytes**

Integers are whole numbers that can have both zero, positive and negative values but no decimal values.

Float -4 bytes

Float and double are used to hold real numbers. The floating point data type allows the user to type decimal values. For example, average marks can be 97.665. if we use int data type, it will strip off the decimal part and print only 97. To print the exact value, we need ‘float’ data type.

Char-1 byte

char stores a single character. Char consists of a single byte.

Double-8 bytes

Double is 8 bytes, which means you can have more precision than float. This is useful in scientific programs that require precision. Float is just a single-precision data type; double is the double-precision data type.

b)

```
#include<stdio.h>
#include<conio.h>
void main()
{
int i,n,m,flag=0; int a[10];
clrscr();
printf("How many elements you want to enter n");
scanf("%d",&n);
printf("Enter element in the array n");
for (i=0; i<n; i++)
scanf("%d", &a[i]);
printf("Enter the element you want to search n");
scanf("%d", &m);
```

Q14		a) Write a C program to find the sum of first and last digit of a number. b) What is type casting? Name the inbuilt typecasting functions available in C language. What is the difference between type casting and type conversion?

ans

a)

```
#include <stdio.h>

int main ()

{

int num, sum=0, firstDigit, lastDigit;
printf ("Enter any number to find sum of first and last digit: ");
scanf("%d", &num);
/* Find last digit to sum */ lastDigit = num % 10;

/* Copy num to first digit */ firstDigit = num;

/* Find the first digit by dividing num by 10 until first digit is left */ while
(num >= 10)

{

num = num / 10;

}

firstDigit = num;

/* Find sum of first and last digit*/ sum = firstDigit + lastDigit;

printf("Sum of first and last digit = %d", sum); return
0;
}
```

b) **Typecasting** refers to the conversion of one data type to another by the user, whereas type conversion refers to the automatic conversion of one type of data to another.atoi() function in C language converts string data type to int data type. itoa () function in C language converts int data type to string data type.The basic difference between type conversion and type casting, i.e. type conversion is made "**automatically**" by compiler whereas, type casting is to be "**explicitly done**" by the programmer. ... When the two types are compatible with each other, then the conversion of one type to other is done automatically by the compiler.

Q15		a) Write a C program to perform linear search on an array of numbers. b) Write a C program to reverse a string without using string handling functions.	

	ans	a) #include <stdio.h>
		
		int main()
		
		{
		
		int array[100], search, c, n;
		
		printf("Enter number of elements in array\n"); scanf("%d", &n);
		printf("Enter %d integer(s)\n", n); for (c = 0; c < n; c++)
		scanf("%d", &array[c]);
		printf("Enter a number to search\n"); scanf("%d", &search);
		for (c = 0; c < n; c++)
		
		{
		
		if (array[c] == search) /* If required element is found */
		
		{
		printf("%d is present at location %d.\n", search, c+1); break;
		
		}
		
		}
		
		if (c == n)
		printf("%d isn't present in the array.\n", search); return 0;
		
		}
		
		b)
		
		#include <stdio.h>
		#include <string.h>
		
		int main()
		
		{
		char Str[100], RevStr[100]; int i, j, len;
		printf("\n Please Enter any String : "); gets(Str);
		
		j = 0;	

OR			
Q16	**a**	**Write a C Program to find the Transpose of a Matrix**	**(7)**

	ans	#include <stdio.h>
		int main() {
		  int a[10][10], transpose[10][10], r, c;
		  printf("Enter rows and columns: ");
		  scanf("%d %d", &r, &c);
		  // asssigning elements to the matrix
		  printf("\nEnter matrix elements:\n");
		  for (int i = 0; i < r; ++i)
		  for (int j = 0; j < c; ++j) {
		    printf("Enter element a%d%d: ", i + 1, j + 1);
		    scanf("%d", &a[i][j]);
		  }
		  // printing the matrix a[][]
		  printf("\nEntered matrix: \n");
		  for (int i = 0; i < r; ++i)
		  for (int j = 0; j < c; ++j) {
		    printf("%d ", a[i][j]);
		    if (j == c - 1)
		    printf("\n");
		  }
		  // computing the transpose
		  for (int i = 0; i < r; ++i)
		  for (int j = 0; j < c; ++j) {
		    transpose[j][i] = a[i][j];
		  }
		  // printing the transpose
		  printf("\nTranspose of the matrix:\n");
		  for (int i = 0; i < c; ++i)
		  for (int j = 0; j < r; ++j) {
		    printf("%d ", transpose[i][j]);
		    if (j == r - 1)
		    printf("\n");
		  }
		  return 0;
		}	

Q16	**b**	**Write a C Program to print number of vowels and consonants in a string**	**(7)**

	ans	#include <stdio.h>
		 int main()
		{
		char str[100];
		int i, vowels, consonants;
		i = vowels = consonants = 0;
		printf("\n Please Enter any String : ");
		gets(str);
		while (str[i] != '\0')
		{
		if(str[i] == 'a' || str[i] == 'e' || str[i] == 'i' || str[i] == 'o' || str[i] == 'u' ||
		str[i] == 'A' || str[i] == 'E' || str[i] == 'I' || str[i] == 'O' || str[i] == 'U')
		{
		vowels++;
		}
		else
		consonants++;
		i++;
		}
		printf("\n Number of Vowels in this String = %d", vowels);
		printf("\n Number of Consonants in this String = %d", consonants); return 0;
		}	

Q17	**a**	**What is the purpose of function declaration and function definition and function call? With examples illustrate their syntax.**	(7)

ans

A function is a group of statements that together performs a task. All C programs are written using functions to improve re-usability and understandability. Parts Of the Function:-

Function Declaration

Declaration of C Function, tells the compiler about a function's name, it's the return type and the parameters. Function declaration in C always ends with a semicolon.In C Language, by default, the return type of a function is an integer(int) data type.A Function declaration is also known as a function prototype.In function declaration name of parameters are not compulsory, but we must define their datatype. Hence the following declaration is also valid.

int getSum(int, int);

Syntax of Declaration of Function

return_type function_name(data_type parameter, data_type parameter)

{

// code to be executed

}

Function Definition

When we define a function, we provide the actual body of the function. A function definition provides the following parts of the function. A function definition specifies the name of the function, the types and number of parameters it expects to receive, and its return type. A function definition also includes a function body with the declarations of its local variables, and the statements that determine what the function does.

Syntax:

returnType functionName(Function arguments){

//body of the function

}

Function Call

Once when we have declared and defined a function, we can call it anywhere in the program as many times. The function will return the value based on our parameters.

Example:-

```
#include<stdio.h>
/* function declaration */int addition();
int main()
{
        //local variable definition
        int answer;

        answer = addition(); //calling a function to get addition value.
        printf("The addition of the two numbers is: %d\n",answer);
        return 0;
}
//function returning the addition of two numbers
int addition()
{
```

Q17	**b)**	**Write a C program to :** **(i) Create a structure containing containing the fields: Name, Price, Quantity, Total Amount.** **(ii) Use separate functions to read and print the data**	(7)

ans

```
#include <stdio.h>
struct item
{
        char  itemName[30];
        int qty;
        float price;
        float amount;
};
/*readItem()- to read values of item and calculate total amount*/
void readItem(struct item *i)
{
        /*read values using pointer*/
        printf("Enter product name: ");
        gets(i->itemName);
        printf("Enter price:");
        scanf("%f",&i->price);
        printf("Enter quantity: ");
        scanf("%d",&i->qty);
        /*calculate total amount of all quantity*/
        i->amount =(float)i->qty * i->price;
}

/*printItem() - to print values of item*/
void printItem(struct item *i)
{
        /*print item details*/
        printf("\nName: %s",i->itemName);
        printf("\nPrice: %f",i->price);
        printf("\nQuantity: %d",i->qty);
        printf("\nTotal Amount: %f",i->amount);
}
int main()
{
        struct item itm;          /*declare variable of structure item*/
        struct item *pItem;       /*declare pointer of structure item*/
        pItem = &itm;/*pointer assignment - assigning address of itm to
pItem*/
        /*read item*/
        readItem(pItem);
        /*print item*/
        printItem(pItem);
        return 0;
}
```

OR

Q18	**a)**	**What are different storage classes in C? Give examples for each.**	(7)
	ans	A storage class defines the scope (visibility) and life-time of variables and/or functions within a C Program.Storage classes in C allocate the storage area of a variable that will be retained in the memory. They are stored in the RAM of a system.There are primarily four storage classes in C, viz. *automatic*, *register*, *static*, and *external* The auto Storage Class Every variable defined in a function or block belongs to *automatic storage class* by default if there is no storage class mentioned. The variables of a function or block belong to the automatic storage class are declared with the auto specifier. Eg:-auto int i = 11; The register Storage Class The variables belonging to a *register storage class* are equivalent to auto in C but are stored in CPU registers and not in the memory, hence the name. They are the ones accessed frequently. The register specifier is used to declare the variable of the register storage class. Variables of a register storage class are local to the block where they are defined and destroyed when the block ends. Eg:-register int i = 10; The static storage class The visibility of static variables is zero outside their function or file, but their values are maintained between calls. The variables with *static storage class* are declared with the static specifier. Static variables are within a function or file. Eg:- static int i; External storage class *External storage class* variables or functions are declared by the 'extern' specifier. When a variable is declared with extern specifier, no storage is allotted to the variable and it is assumed that it has been already defined elsewhere in the program. With an extern specifier, the variable is not initialized.	
Q18	**b)**	**Write a C program to find sum and average of an array of integers using user defined functions.**	(7)

ans

```
#include<stdio.h>
/* Function prototype */
float average(float a[100], int n);
int main()
{
 float a[100], res;
 int i, n;
 clrscr();
 printf("Enter n:\n");
 scanf("%d", &n);
 /* Reading array */
 for(i=0;i< n;i++)
  {
  printf("a[%d]=",i);
  scanf("%f", &a[i]);
  }
 /* Function Call */
 res = average(a,n);
 printf("Average = %f", res);
  return 0;
}
/* Function definition */
float sum_average(float a[10], int n)
{
 int i;
 float sum=0.0;
 for(i=0;i< n;i++)
  {
  sum = sum + a[i];
  }
printf("Sum = %d",sum);
 return(sum/n);

}
```

Q19	**a)**	**Explain the different modes of operations performed on a file in C language.**	**(7)**

	ans	"r"(read) – Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the first character in it. If the file cannot be opened fopen( ) returns NULL. "w"(write) – Searches file. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file. "a"(append) – Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file. "r+" – Searches file. If is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the first character in it. Returns NULL, if unable to open the file. "w+" – Searches file. If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open file. "a+" – Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.	
Q19	**b)**	**Write a program in C to copy the contents of one file into another.**	**(7)**

ans

```
#include <stdio.h>
#include <stdlib.h> // For exit()
int main()
{
   FILE *fptr1, *fptr2;
   char filename[100], c;

   printf("Enter the filename to open for reading \n");
   scanf("%s", filename);

   // Open one file for reading
   fptr1 = fopen(filename, "r");
   if (fptr1 == NULL)
   {
      printf("Cannot open file %s \n", filename);
      exit(0);
   }
    printf("Enter the filename to open for writing \n");
   scanf("%s", filename);
  // Open another file for writing
   fptr2 = fopen(filename, "w");
   if (fptr2 == NULL)
   {
      printf("Cannot open file %s \n", filename);
      exit(0);
   }
   // Read contents from file
   c = fgetc(fptr1);
   while (c != EOF)
   {
      fputc(c, fptr2);
      c = fgetc(fptr1);
   }
   printf("\nContents copied to %s", filename);
   fclose(fptr1);
   fclose(fptr2);
   return 0;
}
```

OR

Q20	**a)**	**Explain how pointers can be passed to functions in C?**	**(7)**

	ans	When we pass a pointer as an argument instead of a variable then the address of the variable is passed instead of the value. So any changes made by the function using the pointer is permanently made at the address of passed variable.
		/* Pass Pointers to Functions in C Sample Program */
		#include<stdio.h>
		void Swap(int *x, int *y)
		{
		int Temp;
		Temp = *x;
		*x = *y;
		*y = Temp;
		}
		int main()
		{
		int a, b;
		printf ("Please Enter 2 Integer Values : ");
		scanf("%d %d", &a, &b);
		printf("\nBefore Swapping A = %d and B = %d", a, b);
		Swap(&a, &b);
		printf(" \nAfter Swapping A = %d and B = %d \n", a, b);
		}	

Q20	**b)**	**Explain any 5 file handling functions in C?**	**(7)**

	ans	1)***fopen() function*** is used to open a file to perform operations such as reading, writing etc. This function takes two arguments. The first argument is a pointer to a string containing name of the file to be opened while the second argument is the mode in which the file is to be opened. The mode can be : 'r' : Open text file for reading. The stream is positioned at the beginning of the file. 'r+' : Open for reading and writing. The stream is positioned at the beginning of the file. 'w' : Truncate file to zero length or create text file for writing. The stream is positioned at the beginning of the file. 'w+' : Open for reading and writing. The file is created if it does not exist, otherwise it is truncated. The stream is positioned at the beginning of the file. 'a' : Open for appending (writing at end of file). The file is created if it does not exist. The stream is positioned at the end of the file. 'a+' : Open for reading and appending (writing at end of file). The file is created if it does not exist. The initial file position for reading is at the beginning of the file, but output is always appended to the end of the file. The fopen() function returns a FILE stream pointer on success while it returns NULL in case of a failure. **2)fread() and fwrite()** The functions fread/fwrite are used for reading/writing data from/to the file opened by fopen function. These functions accept three arguments. The first argument is a pointer to buffer used for reading/writing the data. The data read/written is in the form of 'nmemb' elements each 'size' bytes long. In case of success, fread/fwrite return the number of bytes actually read/written from/to the stream opened by fopen function. 3)The fseek() function is used to set the file position indicator for the stream to a new position. This function accepts three arguments. The first argument is the FILE stream pointer returned by the fopen() function. The second argument 'offset' tells the amount of bytes to seek. The third argument 'whence' tells from where the seek of 'offset' number of bytes is to be done. The available values for whence are SEEK_SET, SEEK_CUR, or SEEK_END. These three values (in order) depict the start of the file, the current position and the end of the file. Upon success, this function returns 0, otherwise it returns -1. 4)The ***fclose() function*** first flushes the stream opened by fopen() and then closes the underlying descriptor. Upon successful completion this function returns 0 else end of file (eof) is returned. 5)***fgets function*** is used to read a file line by line. In a C program, we use fgets function as below. **fgets** (buffer, size, fp); where, buffer – buffer to put the data in.	